

claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf\_416cc0e5-a35\agent-a3fc8670bfff07456.jsonl`
- SHA-256: `dbb0a0c98156dcf4720aed3df994e3da60f2aca66eb29d66a4358fa5fe6cc53a`
- Source modified: `2026-06-22T09:49:31+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf\_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:48:47.611Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:

```
{
  "title": "rally_seg_eval.score_intervals() returns dict with tol_f1/f1@0.5/f1@0.25 keys as expected",
  "file": "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/eval/rally_seg_eval.py:78-123, nightly_reelcount.py:370-372",
  "severity": "low",
  "detail": "Line 370-372 in nightly_reelcount.py calls score_intervals(intervals, gts) and extracts keys ('tol_f1', 'f1@0.5', 'f1@0.25', 'tol_precision', 'tol_recall'). The return dict at rally_seg_eval.py:109-123 supplies all these keys. The extraction at line 371-372 uses a conditional .get(k) for k in list, so missing keys default to None gracefully.",
  "fix": "No fix needed. Quality metrics extraction is safe."
}
```

2. assistant (2026-06-22T09:48:49.624Z)

I'll review this finding by examining the code files mentioned to verify whether the metric extraction is truly safe and returns the expected keys.

3. user (2026-06-22T09:48:50.983Z)

```
1      """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM
nightly).
2
3      Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a
deployment serves
4      it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does
not change**
5      vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
of CACHED
```

```

6      trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real
model*, so it
7      needs a GPU ? hence it runs on an ephemeral GCP GPU VM
(``deploy/gcp/nightly_regression/``), emails the
8      owner the report, and self-deletes. It is **default-additive**: a tripwire, not the
promotion gate.
9
10     What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11     ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12     Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13     **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see $guard).
14     ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15     ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16     ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18     Extensible pattern (add a video = one manifest row + re-freeze):
19     ``eval_baselines/reelcount_manifest.json`` ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20     ``eval_baselines/reelcount_baseline.json`` ? the frozen expected counts (regenerated,
never hand-edited)
21
22     Usage
23     -----
24     Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25     python scripts/nightly_reelcount.py --update-baseline
26
27     Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28     python scripts/nightly_reelcount.py --email
29
30     Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31     (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
changed ? re-freeze).
32     """
33     from __future__ import annotations
34
35     import argparse
36     import datetime as _dt
37     import json
38     import os
39     import sys
40     from dataclasses import dataclass, field
41     from typing import Any, Dict, List, Optional, Tuple
42
43     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
44     if REPO_ROOT not in sys.path:
45         sys.path.insert(0, REPO_ROOT)
46
47     # billing is pure (typing only) ? safe to import at module top.
48     from backend import billing # noqa: E402
49
50     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
51     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
52     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")
53
54     #: Quality metrics we report + (when labels exist) gate ? higher is better, small

```

```

tolerance.
55     QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
56     DEFAULT_QUALITY_TOL = 0.02      # quality is float/abeled ? a looser tripwire than the
exact reel-count
57     DEFAULT_FX_INR_PER_USD = 83.0
58     #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-
south1, ~mid-2026).
59     DEFAULT_MACHINE_USD_PER_HR = 0.35
60
61
62     @dataclass(frozen=True)
63     class Tolerances:
64         quality_tol: float = DEFAULT_QUALITY_TOL
65
66         def to_dict(self) -> Dict[str, Any]:
67             return {"quality_tol": self.quality_tol}
68
69
70     # ----- #
71     # Pure cost math (wraps backend.billing ? no IO).
72     # ----- #
73     def cost_report(billed_seconds: float, machine_usd_per_hr: float,
74                     fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
75         """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
arithmetic).
76
77         ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
``--vm-uptime-seconds``;
78         otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
for boot/install)."""
79         rate_per_sec = float(machine_usd_per_hr) / 3600.0
80         usage = {billing.WALL_SECONDS: float(billed_seconds)}
81         rates = {billing.WALL_SECONDS: rate_per_sec}
82         usd, breakdown = billing.compute_cost_usd(usage, rates)
83         return {
84             "billed_seconds": round(float(billed_seconds), 1),
85             "gpu_seconds": round(float(gpu_seconds), 1),
86             "machine_usd_per_hr": float(machine_usd_per_hr),
87             "usd": usd,
88             "inr": billing.to_inr(usd, fx_inr_per_usd),
89             "breakdown": breakdown,
90         }
91
92
93     # ----- #
94     # Pure summary + comparison core (no IO ? unit-testable with plain dicts).
95     # ----- #
96     def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
97                           cost: Dict[str, Any]) -> Dict[str, Any]:
98         """Compact, comparable snapshot from per-video run results.
99
100         Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
wall_seconds}``.
101         A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it,
never hides it)."""
102         per_video: Dict[str, Any] = {}
103         for r in run_results:
104             per_video[r["name"]] = {
105                 "reel_count": r.get("reel_count"),
106                 "ok": bool(r.get("ok", False)),
107                 "error": r.get("error"),
108                 "quality": r.get("quality"),
109             }
110         return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
111               "n": len(per_video)}

```

```

112
113
114 @dataclass
115 class Finding:
116     video: str
117     metric: str          # "reel_count" | "error" | a quality key | "segmenter" |
"corpus"
118     kind: str           # "regression" | "improvement" | "stale"
119     baseline: Optional[float] = None
120     current: Optional[float] = None
121     detail: str = ""
122
123     def message(self) -> str:
124         if self.kind == "stale":
125             return f"[STALE] {self.metric}: {self.detail}"
126         if self.metric == "error":
127             return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
128         b = "?" if self.baseline is None else f"{self.baseline:g}"
129         c = "?" if self.current is None else f"{self.current:g}"
130         tag = "REGRESSION" if self.kind == "regression" else "improvement"
131         return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"
132
133
134 @dataclass
135 class NightlyResult:
136     findings: List[Finding] = field(default_factory=list)
137     added: List[str] = field(default_factory=list)
138     removed: List[str] = field(default_factory=list)
139
140     @property
141     def regressions(self) -> List[Finding]:
142         return [f for f in self.findings if f.kind == "regression"]
143
144     @property
145     def improvements(self) -> List[Finding]:
146         return [f for f in self.findings if f.kind == "improvement"]
147
148     @property
149     def stale(self) -> List[Finding]:
150         return [f for f in self.findings if f.kind == "stale"]
151
152     def overall_status(self) -> str:
153         if self.regressions:
154             return "REGRESSION"
155         if self.stale:
156             return "STALE"
157         return "PASS"
158
159     def exit_code(self) -> int:
160         if self.regressions:
161             return 1
162         if self.stale:
163             return 4
164         return 0
165
166
167     def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) ->
NightlyResult:
168         """Diff current run vs frozen baseline.
169
170         * Segmenter changed ? STALE (numbers not comparable; re-freeze).
171         * Corpus (video set) changed ? STALE + report added/removed; still diff the
INTERSECTION.
172         * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR =
regression;

```

```

173         quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
174     """
175     res = NightlyResult()
176     if baseline.get("segmenter") != current.get("segmenter"):
177         res.findings.append(Finding("", "segmenter", "stale",
178                                   detail=f"{baseline.get('segmenter')} ?
{current.get('segmenter')}}"))
179     return res
180
181     b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
182     res.added = sorted(set(c_pv) - set(b_pv))
183     res.removed = sorted(set(b_pv) - set(c_pv))
184     if res.added or res.removed:
185         res.findings.append(Finding("", "corpus", "stale",
186                                   detail=f"added={res.added} removed={res.removed}"))
187
188     for v in sorted(set(b_pv) & set(c_pv)):
189         b, c = b_pv[v], c_pv[v]
190         # 1) pipeline must still succeed
191         if not c.get("ok", False) or c.get("reel_count") is None:
192             res.findings.append(Finding(v, "error", "regression",
193                                       detail=str(c.get("error") or "no reel_count
produced")))
194             continue
195         # 2) reel count ? EXACT
196         bc, cc = b.get("reel_count"), c.get("reel_count")
197         if bc is not None and cc is not None and cc != bc:
198             res.findings.append(Finding(v, "reel_count", "regression", float(bc),
199                                       float(cc)))
200
201         # 3) quality metrics ? tolerance (only if both sides have them)
202         bq, cq = b.get("quality") or {}, c.get("quality") or {}
203         for m in QUALITY_HIGHER:
204             bv, cv = bq.get(m), cq.get(m)
205             if bv is None or cv is None:
206                 continue
207             if cv < bv - tols.quality_tol:
208                 res.findings.append(Finding(v, m, "regression", bv, cv))
209             elif cv > bv + tols.quality_tol:
210                 res.findings.append(Finding(v, m, "improvement", bv, cv))
211
212     return res
213
214 # ----- #
215 # Report formatting (pure).
216 # ----- #
217 def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
218     if not qd or qd.get(key) is None:
219         return "?"
220     return f"{qd[key]:.3f}"
221
222 def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
223                  result: NightlyResult, date: str, head: str) -> str:
224     status = result.overall_status()
225     badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
226             "STALE": "? STALE (re-freeze the baseline)"}[status]
227     cost = current.get("cost", {})
228     L: List[str] = [
229         f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
230         "",
231         f"***Status: {badge}** · exit code {result.exit_code()} · segmenter
`{current.get('segmenter')}`",
232         "",
233         f"- HEAD `{head}` · baseline `{baseline.get('git_commit', '?')}` (frozen
{baseline.get('generated_at', '?')})",

```

```

233         f"- **Run cost: ${cost.get('usd', 0):.4f} ({cost.get('inr', 0):.2f})** . "
234         f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
0)}/hr",
235         "",
236     ]
237     if result.regressions:
238         L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions]
+ [""]
239     if result.stale:
240         L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in
result.stale]
241         L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after
confirming the change is intended).", ""]
242
243     # Per-video table.
244     b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
245     L += ["## Per-video", ""]
246         "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
247         "|---|---|---|---|---|"
248     for v in sorted(set(b_pv) | set(c_pv)):
249         b, c = b_pv.get(v, {}), c_pv.get(v, {})
250         bc = b.get("reel_count")
251         cc = c.get("reel_count")
252         rc = f"'?' if bc is None else bc'?'{'ERROR' if not c.get('ok', True) else ('?'
if cc is None else cc)}"
253         bq, cq = b.get("quality"), c.get("quality")
254         tf = f"_{q(bq, 'tol_f1')}_?_{q(cq, 'tol_f1')}"
255         f5 = f"_{q(bq, 'f1@0.5')}_?_{q(cq, 'f1@0.5')}"
256         vstatus = "?" if any(f.video == v and f.kind == "regression" for f in
result.findings) else ""
257         L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
258     L.append("")
259
260     if result.improvements:
261         L += ["_Improvements over baseline (re-freeze to ratchet up):_" \
+ [f"- {f.message()}" for f in result.improvements] + [""]
262     L += ["---",
263         "_Full configured+checkpointed pipeline on the real model (GCP GPU VM).
Tripwire only ? does "
264         "not change the promotion gate. Complements PR #202's cached-trajectory CPU
re-score._"]
265     return "\n".join(L)
266
267
268
269     # ----- #
270     # IO shell ? running the configured pipeline per video (needs backend + GPU + video).
271     # ----- #
272     def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
273         """Load ground-truth rally intervals from a golden-features JSON (`gts` key) or a
rallies CSV
274         (`start,end` columns / pairs). Returns None when no labels are configured."""
275         if not labels_ref:
276             return None
277         from backend.storage import fetch
278         local = fetch(labels_ref)
279         if local.endswith(".json"):
280             with open(local, encoding="utf-8") as fh:
281                 data = json.load(fh)
282                 return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
283         # CSV: header start,end (or first two numeric columns)
284         import csv
285         out: List[Tuple[float, float]] = []
286         with open(local, encoding="utf-8") as fh:
287             for row in csv.reader(fh):
288                 try:

```

```

289         out.append((float(row[0]), float(row[1])))
290     except (ValueError, IndexError):
291         continue
292     return out
293
294
295     def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
296         """(start, end) pairs from segmenter result dicts, tolerant of the two key
conventions in the
297         codebase (``start_time``/``end_time`` ? motion/DB shape ? or ``start``/``end``).
Used only for the
298         optional quality metrics; the reel COUNT is ``len(segs)`` regardless, so a key
mismatch degrades
299         quality scoring gracefully (skipped) without ever miscounting reels."""
300         out: List[Tuple[float, float]] = []
301         for s in segs:
302             start = s.get("start_time", s.get("start"))
303             end = s.get("end_time", s.get("end"))
304             if start is not None and end is not None:
305                 out.append((float(start), float(end)))
306         return out
307
308
309     def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
310                      rerun_on_mismatch: bool = True,
311                      baseline_count: Optional[int] = None) -> Dict[str, Any]:
312         """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.
313
314         The re-run-once guard (the one known non-determinism ? a transient
``add_play_segment``?None drop,
315         database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
316         reproduce, a real change will."""
317         import random
318         import time
319
320         from backend.eval import rally_seg_eval
321         from backend.infrastructure.database import Database
322         from backend.pipeline.segmenters import segmenter_registry
323         from backend.results import Failure
324         from backend.storage import fetch
325         from backend.utils.hashing import compute_video_id
326
327         name = video["name"]
328         try:
329             local = fetch(video["uri"])
330         except Exception as e: # noqa: BLE001 ? surface, never hide
331             return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
332                    "quality": None, "wall_seconds": 0.0}
333
334         gts = _load_gts(video.get("labels_uri"))
335         seg_cls = segmenter_registry.get(segmenter)
336         if not seg_cls:
337             return {"name": name, "reel_count": None, "ok": False,
338                    "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}
339
340     def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
341         random.seed(0) # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
342         db = Database(db_path)
343         video_id = compute_video_id(local)

```

```

344         t0 = time.monotonic()
345         result = seg_cls(db, config).process_video(video_id, local)
346         wall = time.monotonic() - t0
347         if isinstance(result, Failure):
348             return None, None, wall, getattr(result, "message", "segmenter failed")
349         segs = result.value if hasattr(result, "value") else result
350         intervals = _segments_to_intervals(segs)
351         return len(segs), intervals, wall, None
352
353     import tempfile
354     total_wall = 0.0
355     with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356         count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357         total_wall += wall
358         if err is not None:
359             return {"name": name, "reel_count": None, "ok": False, "error": err,
360                    "quality": None, "wall_seconds": round(total_wall, 2)}
361         # re-run-once guard for the transient DB-drop flake
362         if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
363             count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
364             total_wall += wall2
365             if err2 is None and count2 is not None:
366                 count, intervals = count2, intervals2 # the reproduced (or corrected)
367
368         quality = None
369         if gts and intervals is not None:
370             row = rally_seg_eval.score_intervals(intervals, gts)
371             quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
372 "tol_precision", "tol_recall")}
373             if k in row}
374
375         return {"name": name, "reel_count": count, "ok": True, "error": None,
376                "quality": quality, "wall_seconds": round(total_wall, 2)}
377
378     def load_manifest(path: str) -> Dict[str, Any]:
379         with open(path, encoding="utf-8") as fh:
380             return json.load(fh)
381
382
383     def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
384                   rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
385         """Run every manifest video through the configured pipeline. Returns (run_results,
386 total_wall_s)."""
387         from backend.config import load_config
388
389         config = load_config()
390         # apply manifest config overrides onto the typed config (dotted keys)
391         for dotted, val in (manifest.get("config_overrides") or {}).items():
392             _set_dotted(config, dotted, val)
393         segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
394 "default_segmenter", "motion")
395         b_pv = (baseline or {}).get("per_video", {})
396         results: List[Dict[str, Any]] = []
397         total_wall = 0.0
398         for v in manifest.get("videos", []):
399             base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
400             r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
401             total_wall += float(r.get("wall_seconds") or 0.0)
402             results.append(r)
403         return results, total_wall
404
405     def _set_dotted(config: Any, dotted: str, value: Any) -> None:

```



```

405     """Best-effort dotted-path set onto a pydantic config (e.g.
'indexing.motion_threshold')."""
406     parts = dotted.split(".")
407     obj = config
408     for p in parts[:-1]:
409         obj = getattr(obj, p, None)
410         if obj is None:
411             return
412     try:
413         setattr(obj, parts[-1], value)
414     except Exception: # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't
crash the run
415         pass
416
417
418     def _git_head() -> str:
419         import subprocess
420         try:
421             out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
422                                 capture_output=True, text=True, timeout=10)
423             return out.stdout.strip() or "?"
424         except Exception: # noqa: BLE001
425             return "?"
426
427
428     def load_baseline(path: str) -> Optional[Dict[str, Any]]:
429         if not os.path.exists(path):
430             return None
431         with open(path, encoding="utf-8") as fh:
432             return json.load(fh)
433
434
435     def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:
436         out = dict(snapshot)
437         out["generated_at"] = _dt.date.today().isoformat()
438         out["git_commit"] = _git_head()
439         out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python
scripts/nightly_reelcount.py "
440                        "--update-baseline` after an INTENDED engine change or when adding a
video. Tied to the "
441                        "configured+checkpointed engine + the manifest corpus.")
442         out.pop("cost", None) # cost is per-run, not part of the frozen baseline
443         os.makedirs(os.path.dirname(path), exist_ok=True)
444         tmp = path + ".tmp"
445         with open(tmp, "w", encoding="utf-8") as fh:
446             json.dump(out, fh, indent=2, sort_keys=True)
447             fh.write("\n")
448         os.replace(tmp, path)
449
450
451     def build_parser() -> argparse.ArgumentParser:
452         p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count +
quality + cost.")
453         p.add_argument("--manifest", default=DEFAULT_MANIFEST)
454         p.add_argument("--baseline", default=DEFAULT_BASELINE)
455         p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
456         p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
457         p.add_argument("--vm-uptime-seconds", type=float, default=None,
458                        help="authoritative VM uptime (boot?delete) for the cost line; "
459                        "defaults to the in-process pipeline wall time (a lower bound)")
460         p.add_argument("--no-rerun-guard", action="store_true",
461                        help="disable the re-run-once-on-mismatch guard (the DB-drop flake
mitigation)")
462         p.add_argument("--update-baseline", action="store_true",
463                        help="freeze the current reel counts + quality as the new baseline

```

```

instead of checking")
464     p.add_argument("--email", action="store_true", help="email the report (see
scripts/nightly_email.py)")
465     p.add_argument("--email-to", default=None, help="override the report recipient")
466     p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
467     p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
468     return p
469
470
471     def main(argv: Optional[List[str]] = None) -> int:
472         import logging
473         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
474         log = logging.getLogger("nightly_reelcount")
475         args = build_parser().parse_args(argv)
476         tols = Tolerances(quality_tol=args.quality_tol)
477         date = args.date or _dt.date.today().isoformat()
478
479         if not os.path.exists(args.manifest):
480             log.error("manifest not found: %s", args.manifest)
481             return 2
482         manifest = load_manifest(args.manifest)
483         baseline = load_baseline(args.baseline)
484
485         try:
486             run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
487             except Exception as e: # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
488                 log.error("run failed: %s", e)
489                 return 2
490             if not run_results:
491                 log.error("no videos in manifest %s", args.manifest)
492                 return 2
493
494             billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
495             cost = cost_report(
496                 billed_seconds=billed,
497                 machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
498                 fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
499                 gpu_seconds=total_wall,
500             )
501             segmenter = manifest.get("segmenter", "motion")
502             snapshot = summarize_results(run_results, segmenter, cost)
503             snapshot["git_commit"] = _git_head()
504             snapshot["generated_at"] = date
505
506             if args.update_baseline:
507                 save_baseline(args.baseline, snapshot)
508                 log.info("baseline frozen ? %s", args.baseline)
509                 for r in run_results:
510                     log.info(" %s: reels=%s ok=%s", r["name"], r.get("reel_count"),
r.get("ok"))
511                 return 0
512
513             if baseline is None:
514                 log.error("no baseline at %s ? freeze one with --update-baseline (on the
configured engine).", args.baseline)
515                 return 3
516
517             result = compare(baseline, snapshot, tols)
518             report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])

```

```

519
520     if not args.no_report:
521         os.makedirs(args.report_dir, exist_ok=True)
522         md_path = os.path.join(args.report_dir, f"{date}.md")
523         with open(md_path, "w", encoding="utf-8") as fh:
524             fh.write(report + "\n")
525         with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8")
as fh:
526             json.dump({"date": date, "status": result.overall_status(), "exit_code":
result.exit_code(),
527                       "snapshot": snapshot, "regressions": [f.message() for f in
result.regressions]},
528                       fh, indent=2, sort_keys=True)
529         log.info("report ? %s", md_path)
530
531     print(report)
532
533     if args.email:
534         try:
535             from scripts.nightly_email import send_report
536             subject = f"[KhelSutra nightly] {result.overall_status()} ? reel-count
engine regression {date}"
537             sent = send_report(subject, report, to=args.email_to)
538             log.info("email %s", "sent" if sent else "skipped (no creds ? report still
written)")
539         except Exception as e: # noqa: BLE001 ? a broken mailer must NOT mask the
regression verdict
540             log.error("email failed (report still written): %s", e)
541
542     code = result.exit_code()
543     if code == 1:
544         log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions),
args.report_dir)
545     elif code == 4:
546         log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-
baseline")
547     else:
548         log.info("nightly reel-count check PASSED (cost $%.4f / ?%.2f)", cost["usd"],
cost["inr"])
549     return code
550
551
552     if __name__ == "__main__":
553         raise SystemExit(main())
554

```

4. user (2026-06-22T09:48:51.505Z)

```

1     """One-command rally-segmentation eval ? score a windowing against the golden labels
with the
2     over-segmentation-sensitive metric set, honestly.
3
4     This is the harness the rally-quality project (`docs/RALLY_QUALITY_RESEARCH.md` $4.5 /
$7)
5     measures every tier against. It closes the loop the over-merge finding exposed: plain
temporal
6     IoU-F1 is blind to merged mega-windows, so we report it alongside segmental F1@k,
the
7     segment-count ratio, and the explicit `merge_split` breakdown
(`segmentation_metrics`).
8
9     For each golden video it: parses the cached WASB/TrackNet trajectory CSV ? builds
candidate
10    windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the
SERVED

```

```

11     operating point and any future windowing the same way) ? scores the windows-as-rallies
against
12     the ground-truth rally intervals. Aggregates per-video held-out F1 with a bootstrap CI,
and can
13     diff two windowings with a paired sign-test + CI (the per-tier ``delta``).
14
15     GROUND TRUTH: the golden manifest (``output/human_lovo_manifest.json``) gives each
video's
16     trajectory path + fps + frame_width; the GT rally intervals come from
17     ``output/<name>_golden_features.json`` (the ``gts`` key) so this needs no extra label
files.
18
19     Pure offline (CPU): trajectory parsing + windowing + interval scoring; no GPU/Gemini.
20     """
21     from __future__ import annotations
22
23     import argparse
24     import json
25     import os
26     from typing import Any, Dict, List, Optional, Tuple
27
28     from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
29     from backend.eval.metrics import Interval
30     from backend.eval.windowing import PRESETS, SERVED, WindowingPreset
31
32     DEFAULT_MANIFEST = os.path.join("output", "human_lovo_manifest.json")
33     DEFAULT_FEATURES_DIR = "output"
34     DEFAULT_IOU = 0.5
35
36     #: Long-rally lens threshold (s). A rally is "long" (an eye-catching highlight) when its
duration
37     #: (end ? start) exceeds this. The local detector over-fires, and its false positives
are mostly
38     #: SHORT (motion blips, background-court fragments on multicourt footage); filtering to
long rallies
39     #: strips most of those FPs and reveals the real product quality on the long-rally happy
path. Used
40     #: as the default split point for the stratified report (``--long-tau`` overrides it).
DURATION, not
41     #: shot-count, is the filter: golden ``shots_count`` is unlabeled and the no-AI path
reports it as
42     #: "Unknown", whereas duration is derived from the same start/end labels we already
trust.
43     LONG_RALLY_TAU = 5.0
44
45     #: Multicourt golden clips ? the court-type lookup for the stratified report. The golden
manifest
46     #: (``output/human_lovo_manifest.json``) is gitignored, so this committed set is the
AUDITABLE source
47     #: of the single-vs-multicourt strata, kept in sync with ``docs/GOLDEN_VIDEOS.md`` (the
clips tagged
48     #: "multicourt"). Multicourt footage carries background games on adjacent courts ? the
dominant
49     #: source of short false positives ? so we report it separately. A manifest entry MAY
carry an
50     #: explicit ``court_type`` field, which overrides this set (see :func:`court_type_for`).
51     MULTICOURT_CLIPS = frozenset({"mahadevpura_2", "GX010128", "Badminton_BXH_2",
"Boxhill_Doubles"})
52
53
54     def court_type_for(video: Dict[str, Any]) -> str:
55         """``multicourt`` or ``single`` for a golden video. Honors an explicit manifest
56         ``court_type`` field when present, else falls back to membership in
``MULTICOURT_CLIPS``
57         (keyed by ``name``), else ``single``."""

```

```

58         ct = video.get("court_type")
59         if isinstance(ct, str) and ct:
60             return ct
61         return "multicourt" if video.get("name") in MULTICOURT_CLIPS else "single"
62
63
64     def filter_by_min_duration(ivs: List[Interval], min_duration: float) -> List[Interval]:
65         """The long-rally lens: keep only intervals strictly longer than ``min_duration``
seconds.
66
67         ``min_duration <= 0`` is an EXACT passthrough (the default-OFF semantics ? every
real rally has
68         positive duration, so nothing is dropped). It's a FILTER, not a new metric: applied
identically
69         to predictions AND ground truth before any matching, so the tIoU and R2 paths stay
consistent."""
70         if min_duration <= 0.0:
71             return list(ivs)
72         return [iv for iv in ivs if (iv[1] - iv[0]) > min_duration]
73
74
75     # ----- #
76     # Pure scoring core (no I/O ? unit-testable)
77     # ----- #
78     def score_intervals(preds: List[Interval], gts: List[Interval],
79                         iou: float = DEFAULT_IOU,
80                         tau_start: Optional[float] = None,
81                         tau_end: Optional[float] = None,
82                         min_duration: float = 0.0) -> Dict[str, Any]:
83         """The full per-video metric row for predicted vs ground-truth rally intervals.
84
85         Pairs temporal-IoU P/R/F1 + boundary error (``metrics.score``) with the
86         over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown)
so a
87         merged mega-window is *visible* even when IoU-F1 isn't moved.
88
89         Also carries the **R2** task-faithful block (``tolerance_metrics``, the headline
going forward;
90         tIoU here is the SECONDARY trend-line per the augment-then-ablation-gate decision):
strict-1:1
91         tolerance-match ``tol_f1/tol_precision/tol_recall`` (over-production costs
precision) + the
92         start/end boundary-error medians + the over-seg guard counts. See
docs/R2_EVAL_METRIC_DESIGN.md.
93
94         ``min_duration`` > 0 applies the **long-rally lens**
(:func:`filter_by_min_duration`): BOTH preds
95         and GTs are filtered to rallies longer than that many seconds BEFORE any matching,
so the whole
96         row (tIoU set AND R2 block) scores only long rallies. Default 0.0 = OFF (exact
passthrough).
97         """
98         from backend.eval import tolerance_metrics as tm
99         preds = filter_by_min_duration(preds, min_duration)
100        gts = filter_by_min_duration(gts, min_duration)
101        ts = tm.TAU_START if tau_start is None else tau_start
102        te = tm.TAU_END if tau_end is None else tau_end
103        sc = metrics.score(preds, gts, iou_threshold=iou)
104        f1k = segmentation_metrics.f1_at_overlaps(preds, gts)
105        ms = segmentation_metrics.merge_split_report(preds, gts)
106        tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
107        be = tm.boundary_error_report(preds, gts) # unconditional best-overlap (the
R4-aiming diagnostic)
108        osr = tm.over_seg_report(preds, gts)
109        return {

```

```

110         "f1": sc.f1, "precision": sc.precision, "recall": sc.recall, "mean_iou":
sc.mean_iou,
111         "mean_start_error": sc.mean_start_error, "mean_end_error": sc.mean_end_error,
112         "f1@0.1": f1k[0.1], "f1@0.25": f1k[0.25], "f1@0.5": f1k[0.5],
113         "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
114         "merge_rate": ms["merge_rate"], "split_rate": ms["split_rate"],
115         "merges": ms["merges"], "splits": ms["splits"], "missed": ms["missed"],
116         "spurious": ms["spurious"], "num_pred": len(preds), "num_gt": len(gts),
117         # --- R2 (tolerance-match) ? the task-faithful headline block ---
118         "tau_start": ts, "tau_end": te,
119         "tol_f1": tol_f1, "tol_precision": tol_p, "tol_recall": tol_r,
120         "tol_matched": float(len(tol_matches)),
121         "dstart_abs_median": be["dstart"]["abs_median"], "dend_abs_median":
be["dend"]["abs_median"],
122         "split_count": osr["split_count"], "merge_count": osr["merge_count"],
123     }
124
125
126     _AGG_KEYS = ("f1", "f1@0.25", "f1@0.5", "merge_rate", "split_rate",
"segment_count_ratio",
127                 "precision", "recall",
128                 "tol_f1", "tol_precision", "tol_recall", "dstart_abs_median",
"dend_abs_median")
129
130
131     def aggregate(rows: List[Dict[str, Any]]) -> Dict[str, Any]:
132         """Aggregate per-video rows: mean (+ bootstrap 95% CI) of the headline metrics
across videos.
133
134         Per-video means (not pooled) keep videos the unit of evidence, matching the
harness's
135         leave-one-video-out discipline (windows within a video are correlated)."""
136         out: Dict[str, Any] = {"n": len(rows)}
137         for k in _AGG_KEYS:
138             vals = [r[k] for r in rows]
139             out[k] = eval_stats.mean_with_ci(vals) if vals else None
140         return out
141
142
143     # ----- #
144     # Golden corpus I/O
145     # ----- #
146     def load_golden(manifest_path: str = DEFAULT_MANIFEST,
147                    features_dir: str = DEFAULT_FEATURES_DIR) -> List[Dict[str, Any]]:
148         """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
rally
149         intervals (`<name>_golden_features.json` `gts`). Videos with no GTs are kept but
flagged."""
150         with open(manifest_path) as fh:
151             manifest = json.load(fh)
152         out: List[Dict[str, Any]] = []
153         for v in manifest:
154             feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
155             gts: List[Interval] = []
156             if os.path.exists(feat):
157                 with open(feat) as fh:
158                     gts = [(float(s), float(e)) for s, e in (json.load(fh).get("gts") or
[])]
159             entry: Dict[str, Any] = {"name": v["name"], "trajectory": v["trajectory"],
160                                     "fps": float(v["fps"]), "frame_width":
float(v["frame_width"]),
161                                     "gts": gts}
162             # Optional prominent-court gate inputs (multicourt G2; default absent ? the gate
is a no-op):
163             # a normalized court_polygon + optional frame_height + court_height_pad from the

```

```

manifest.
164         for k in ("court_polygon", "frame_height", "court_height_pad"):
165             if k in v:
166                 entry[k] = v[k]
167             out.append(entry)
168         return out
169
170
171     def windows_for(video: Dict[str, Any], preset: WindowingPreset,
172                    min_crossings: int = 0) -> List[Interval]:
173         """Parse a video's trajectory, window it under ``preset`` ? predicted rally
intervals.
174
175         ``min_crossings`` > 0 applies the net-crossings rally-state gate (Gate-A,
176         ``rally_gate.gate_windows``): drop windows whose shuttle crosses the net fewer than
this many
177         times (a non-rally fragment). This is the cheapest rally-state cue (W2) ? CPU-only,
derived
178         from the trajectory + an auto-estimated net axis; it cleans up the over-split that
bounded
179         windowing (W1) leaves behind.
180
181         NEVER-EMPTY safeguard (mirrors ``FusionSegmenter._select_for_handoff``): if the gate
would
182         drop EVERY window of a video that had ?1, fall back to the ungated windows. The gate
is
183         strictly *pruning* ? it never zeroes out a non-empty video ? so W2-on-W1 is "do no
harm".
184         """
185         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
186         points = TrackNetRunner.parse_trajectory_csv(video["trajectory"])
187         wins = windowing.build_windows(points, video["fps"], video["frame_width"], preset)
188         ivs = windowing.windows_to_intervals(wins)
189         # Prominent-court WINDOW gate (multicourt G2, default-OFF): when the video carries a
normalized
190         # court_polygon, drop candidate rallies whose shuttle is MAJORITY outside the
prominent court ? a
191         # background/adjacent-court rally. Window-level (not point-level): a background
rally's central
192         # tail keeps a few in-court points, enough to hold a window, so point-clearing alone
is too leaky
193         # (measured on GX010142 rally #1). No polygon ? no-op. See
docs/PROMINENT_COURT_DETECTION.md.
194         poly = video.get("court_polygon")
195         if poly and ivs:
196             from backend.pipeline.detectors.court_gate import gate_intervals_to_court
197             fw = float(video["frame_width"])
198             fh = float(video.get("frame_height") or fw * 9.0 / 16.0)
199             ivs = gate_intervals_to_court(
200                 ivs, points, poly, float(video["fps"]), fw, fh,
201                 min_in_court_frac=float(video.get("court_min_in_court_frac", 0.5)),
202                 height_pad=float(video.get("court_height_pad", 0.0) or 0.0))
203         if min_crossings > 0 and ivs:
204             from backend.pipeline.detectors.rally_gate import gate_windows
205             gated = [(g.start, g.end)
206                     for g in gate_windows(points, ivs, video["fps"],
min_crossings=min_crossings)
207                     if g.kept]
208             # Gating emptied a non-empty video ? keep the ungated windows.
209             ivs = gated if gated else ivs
210         return ivs
211
212
213     #: One windowed video before scoring: (name, court_type, predicted intervals, GT
intervals).

```

```

214     PredictedVideo = Tuple[str, str, List[Interval], List[Interval]]
215
216
217     def predict_all(golden: List[Dict[str, Any]], preset: WindowingPreset,
218                     min_crossings: int = 0) -> List["PredictedVideo"]:
219         """Window every scorable golden video ONCE (the expensive trajectory-parse +
windowing step),
220         returning ``(name, court_type, preds, gts)`` per video. Lets callers re-score the
same
221         predictions under several lenses (e.g. the all-vs-long stratified report) without
re-windowing."""
222         out: List[PredictedVideo] = []
223         for v in golden:
224             if not v["gts"] or not os.path.exists(v["trajectory"]):
225                 continue
226             out.append((v["name"], court_type_for(v), windows_for(v, preset, min_crossings),
v["gts"]))
227         return out
228
229
230     def _score_predicted(predicted: List["PredictedVideo"], iou: float,
231                           tau_start: Optional[float], tau_end: Optional[float],
232                           min_duration: float) -> List[Dict[str, Any]]:
233         """Score pre-windowed videos into per-video rows (carrying ``name`` +
``court_type``)."""
234         rows: List[Dict[str, Any]] = []
235         for name, court, preds, gts in predicted:
236             row = score_intervals(preds, gts, iou, tau_start, tau_end, min_duration)
237             row["name"] = name
238             row["court_type"] = court
239             rows.append(row)
240         return rows
241
242
243     def evaluate(golden: List[Dict[str, Any]], preset: WindowingPreset,
244                 iou: float = DEFAULT_IOU, min_crossings: int = 0,
245                 tau_start: Optional[float] = None, tau_end: Optional[float] = None,
246                 min_duration: float = 0.0) -> Dict[str, Any]:
247         """Score every golden video under ``preset`` (+ optional net-crossings gate); return
per-video
248         rows + the aggregate. Each row carries both the tIoU set and the R2 tolerance-match
block, plus
249         its ``court_type``. ``min_duration`` > 0 applies the long-rally lens (preds + GTs
filtered to
250         rallies longer than that many seconds) consistently across both metric paths."""
251         rows = _score_predicted(predict_all(golden, preset, min_crossings),
252                                 iou, tau_start, tau_end, min_duration)
253         return {"preset": preset.to_dict(), "iou": iou, "min_crossings": min_crossings,
254                 "min_duration": min_duration, "rows": rows, "aggregate": aggregate(rows)}
255
256
257     def compare(golden: List[Dict[str, Any]], base: WindowingPreset, cand: WindowingPreset,
258                 iou: float = DEFAULT_IOU, base_min_crossings: int = 0,
259                 cand_min_crossings: int = 0,
260                 tau_start: Optional[float] = None, tau_end: Optional[float] = None,
261                 guard_weight_merge: Optional[float] = None,
262                 guard_weight_split: Optional[float] = None,
263                 guard_use_rates: bool = True, min_duration: float = 0.0) -> Dict[str, Any]:
264         """Baseline vs candidate (windowing and/or net-crossings gate): per-video paired ?
(+ CI +
265         sign-test) on both tIoU-F1 and the R2 ``tol_f1`` ? the per-tier 'number'. Also
reports ? on the
266         over-segmentation metrics AND the **R1 net over-seg promotion guard**
(``over_seg_guard``):
267         the honest promote/block verdict for cand-over-base, with the strict per-video rule

```



```

reported
268         alongside so the refinement is auditable. ``min_duration`` > 0 scores both sides
through the
269         long-rally lens (same filter applied to base and cand ? it's a scoring lens, not a
windowing
270         knob, so it is identical on both sides of the A/B)."""
271         from backend.eval import tolerance_metrics as tm
272         b = evaluate(golden, base, iou, base_min_crossings, tau_start, tau_end,
min_duration)
273         c = evaluate(golden, cand, iou, cand_min_crossings, tau_start, tau_end,
min_duration)
274         by_name_b = {r["name"]: r for r in b["rows"]}
275         paired = [(by_name_b[r["name"]], r) for r in c["rows"] if r["name"] in by_name_b]
276         delta: Dict[str, Any] = {}
277         for k in ("f1", "f1@0.25", "tol_f1", "merge_rate", "split_rate",
"segment_count_ratio"):
278             delta[k] = eval_stats.paired_delta_ci([rb[k] for rb, _ in paired],
279                                                    [rc[k] for _, rc in paired])
280         gkw: Dict[str, Any] = {"use_rates": guard_use_rates}
281         if guard_weight_merge is not None:
282             gkw["weight_merge"] = guard_weight_merge
283         if guard_weight_split is not None:
284             gkw["weight_split"] = guard_weight_split
285         guard = tm.over_seg_guard([rb for rb, _ in paired], [rc for _, rc in paired], **gkw)
286         return {"base": b, "cand": c, "n_paired": len(paired), "delta": delta,
"over_seg_guard": guard}
287
288
289     def stratified_report(golden: List[Dict[str, Any]], preset: WindowingPreset,
290                          iou: float = DEFAULT_IOU, min_crossings: int = 0,
291                          tau_start: Optional[float] = None, tau_end: Optional[float] =
None,
292                          long_tau: float = LONG_RALLY_TAU) -> Dict[str, Any]:
293         """The long-rally lens payoff: ``{all vs long(>long_tau)} × {single vs multicourt}``
aggregates.
294
295         The thesis: local OVER-FIRES, and its false positives are mostly SHORT (motion
blips,
296         background-court fragments on multicourt footage). Filtering to long rallies should
strip those
297         FPS ? precision (``tol_precision``) should jump, most on multicourt. This table is
how we read
298         "do we beat Gemini on the single-match long-rally happy path?".
299
300         Windowing is run ONCE per video (``predict_all``); each cell re-scores the same
predictions
301         under its duration lens. A cell aggregates only videos with >=1 golden rally in that
stratum
302         (``num_gt`` > 0): a clip with no long rally is not evidence about long-rally quality
and its
303         0/0 recall would distort the mean. ``n`` per cell reports the videos that survived
that gate."""
304         predicted = predict_all(golden, preset, min_crossings)
305         long_label = f"long>{long_tau:g}s"
306         strata = (("all", 0.0), (long_label, long_tau))
307         courts = ("single", "multicourt", "all")
308         cells: Dict[Tuple[str, str], Dict[str, Any]] = {}
309         for dur_label, min_dur in strata:
310             rows = _score_predicted(predicted, iou, tau_start, tau_end, min_dur)
311             for court in courts:
312                 sub = [r for r in rows
313                        if (court == "all" or r["court_type"] == court) and r["num_gt"] > 0]
314                 cells[(dur_label, court)] = aggregate(sub)
315         return {"long_tau": long_tau, "labels": [s[0] for s in strata], "courts":
list(courts),

```

```

316         "cells": cells, "preset": preset.to_dict(), "iou": iou, "min_crossings":
min_crossings}
317
318
319     # ----- #
320     # Reporting
321     # ----- #
322     def _ci(m: Optional[Dict[str, Any]]) -> str:
323         return "?" if not m else f"{m['mean']:.3f} [{m['ci_low']:.3f},{m['ci_high']:.3f}]"
324
325
326     def _format_guard(g: Dict[str, Any]) -> str:
327         """Render the R1 net over-seg promotion-guard verdict (with the strict rule for
contrast)."""
328         basis = "rate" if g["use_rates"] else "count"
329         lines = [f"\n## R1 over-seg promotion guard ({basis}-based, "
330                 f"w_merge={g['weight_merge']}, w_split={g['weight_split']}",
331                 f"NET verdict: {'PASS' if g['passes'] else 'BLOCK'} "
332                 f"(net cost {g['base_net']:.3f} ? {g['cand_net']:.3f}, ?
{g['net_delta']:+.3f}; "
333                 f"no_merge_rate_regress={g['no_merge_rate_regress']})",
334                 f"strict per-video rule (R2 §2.3.3, for contrast): "
335                 f"{'pass' if g['strict_passes'] else 'BLOCK'} "
336                 f"(count increases ? merges:{len(g['new_merges'])})
splits:{len(g['new_splits'])})"]
337         if g["merge_rate_regress"]:
338             names = ", ".join(f"{r['name']}({r['base']:.2f}?{r['cand']:.2f})"
339                               for r in g["merge_rate_regress"])
340             lines.append(f" ? merge_rate REGRESSED on: {names}")
341         return "\n".join(lines)
342
343
344     def format_report(result: Dict[str, Any]) -> str:
345         p = result["preset"]
346         bounded = []
347         if p.get("max_window_duration"):
348             bounded.append(f"cap={p['max_window_duration']} + ((hard)" if
p.get("hard_cap") else ""))
349         if p.get("inactivity_end"):
350             bounded.append(f"inact_end={p['inactivity_end']}/rt={p['inactivity_rally_thresh']}")
351             f"/md={p['inactivity_min_density']}")
352         if p.get("blob_fallback"):
353             bounded.append("blob_fallback")
354         extra = (" " + ", ".join(bounded)) if bounded else ""
355         md = result.get("min_duration", 0.0) or 0.0
356         md_tag = f", LONG-RALLY LENS min_dur>{md:g}s" if md > 0 else ""
357         lines = [f"# Rally-segmentation eval ? windowing '{p['name']}' "
358                 f"(vt={p['velocity_thresh']}, merge_gap={p['merge_gap']}, "
359                 f"min_win={p['min_window_duration']}{extra}), IoU={result['iou']}{md_tag}",
360                 ""]
361         lines.append("| video | F1 | F1@0.25 | F1@0.5 | merges | merge_rate | seg_ratio |
pred/gt |")
362         lines.append("|---|---|---|---|---|---|---|")
363         for r in result["rows"]:
364             lines.append(f"| {r['name']} | {r['f1']:.3f} | {r['f1@0.25']:.3f} |
{r['f1@0.5']:.3f} | "
365                           f"{int(r['merges'])} | {r['merge_rate']:.2f} |
{r['segment_count_ratio']:.2f} | "
366                           f"{r['num_pred']}/{r['num_gt']} |")
367         a = result["aggregate"]
368         lines += [f"***Aggregate (n={a['n']}, mean [95% CI]):** "
369                  f"F1 {_ci(a['f1'])} · F1@0.25 {_ci(a['f1@0.25'])} · F1@0.5
_ci(a['f1@0.5'])} · "
370                  f"merge_rate {_ci(a['merge_rate'])} · seg_ratio

```

```

{_ci(a['segment_count_ratio'])}}"]
370     # --- R2 (tolerance-match) block ? the task-faithful headline (tIoU above is
secondary) ---
371     rows = result["rows"]
372     if rows and "tol_f1" in rows[0]:
373         ts, te = rows[0]["tau_start"], rows[0]["tau_end"]
374         lines += ["", f"### R2 ? tolerance-match (?_start={ts}s, ?_end={te}s, strict
1:1)", "",
375                 "| video | tolF1 | tolP | tolR | |?start| | |?end| | split | merge |",
376                 "|---|---|---|---|---|---|---|---|"]
377     for r in rows:
378         lines.append(f"| {r['name']} | {r['tol_f1']:.3f} | {r['tol_precision']:.2f}
| "
379                     f"{r['tol_recall']:.2f} | {r['dstart_abs_median']:.2f} | "
380                     f"{r['dend_abs_median']:.2f} | {int(r['split_count'])} | "
381                     f"int(r['merge_count'])} |")
382         lines += ["", f"***R2 aggregate (n={a['n']})** tolF1 {_ci(a['tol_f1'])} . "
383                 f"tolP {_ci(a['tol_precision'])} . tolR {_ci(a['tol_recall'])} . "
384                 f"|?start| {_ci(a['dstart_abs_median'])} . |?end|
{_ci(a['dend_abs_median'])} "
385                 f"(start?solved / end=the gap)"]
386     return "\n".join(lines)
387
388 def format_stratified(strat: Dict[str, Any]) -> str:
389     """Render the long-rally stratified table: {all vs long} x {single, multicourt, all-
courts}.
390
391     ``tolP`` is the column that carries the story ? if filtering to long rallies strips
local's
392     short false positives, precision rises (most on multicourt). ``n`` is the videos
with >=1 golden
393     rally in that stratum."""
394     long_tau = strat["long_tau"]
395     cells = strat["cells"]
396     lines = [f"\n## Long-rally lens ? stratified (long = duration > {long_tau:g}s; "
397             "n = videos with >=1 golden rally in the stratum)", "",
398             "| stratum | court | n | tolF1 | tolP | tolR | tIoU F1@0.5 | seg_ratio |",
399             "|---|---|---|---|---|---|---|---|"]
400     for dur_label in strat["labels"]:
401         for court in strat["courts"]:
402             a = cells[(dur_label, court)]
403             lines.append(f"| {dur_label} | {court} | {a['n']} | {_ci(a['tol_f1'])} | "
404                         f"{_ci(a['tol_precision'])} | {_ci(a['tol_recall'])} | "
405                         f"{_ci(a['f1@0.5'])} | {_ci(a['segment_count_ratio'])} |")
406     return "\n".join(lines)
407
408
409 def _resolve_preset(name: str) -> WindowingPreset:
410     if name in PRESETS:
411         return PRESETS[name]
412     raise SystemExit(f"unknown preset '{name}'; choices: {sorted(PRESETS)}")
413
414
415 def _build_preset(args: argparse.Namespace, side: str) -> WindowingPreset:
416     """Resolve the ``base`` (``--preset``) or ``cand`` (``--vs``) windowing from
``args`` by
417     applying the ``dataclasses.replace`` overrides for cap / hard-cap / blob-fallback /
inactivity. ``cand`` falls back to the base override when its ``--vs`` variant is
unset
418
419     (so a baseline-vs-candidate promotion check runs in one shot)."""
420     from dataclasses import replace
421     if side == "base":
422         preset = _resolve_preset(args.preset)
423         cap = args.max_window_duration

```

```

424         hard_cap = args.hard_cap
425         blob_fallback = args.blob_fallback
426         inactivity_end = args.inactivity_end
427     else:
428         preset = _resolve_preset(args.vs)
429         cap = (args.vs_max_window_duration if args.vs_max_window_duration is not None
430               else args.max_window_duration)
431         hard_cap = args.vs_hard_cap or args.hard_cap
432         blob_fallback = args.vs_blob_fallback or args.blob_fallback
433         inactivity_end = (args.vs_inactivity_end if args.vs_inactivity_end is not None
434                           else args.inactivity_end)
435     if cap is not None:
436         preset = replace(preset, max_window_duration=cap)
437     if hard_cap:
438         preset = replace(preset, hard_cap=True)
439     if blob_fallback:
440         preset = replace(preset, blob_fallback=True)
441     if args.blob_factor is not None:
442         preset = replace(preset, blob_factor=args.blob_factor)
443     if inactivity_end is not None:
444         preset = replace(preset, inactivity_end=inactivity_end)
445     if args.inactivity_rally_thresh is not None:
446         preset = replace(preset, inactivity_rally_thresh=args.inactivity_rally_thresh)
447     if args.inactivity_min_density is not None:
448         preset = replace(preset, inactivity_min_density=args.inactivity_min_density)
449     return preset
450
451
452     def main() -> None:
453         ap = argparse.ArgumentParser(description="Rally-segmentation eval vs golden labels
454 (offline).")
455         ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
456         ap.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR)
457         ap.add_argument("--preset", default=SERVED.name, help="windowing preset (default:
458 served)")
459         ap.add_argument("--vs", default=None, help="second preset to diff against --preset")
460         ap.add_argument("--iou", type=float, default=DEFAULT_IOU)
461         ap.add_argument("--min-crossings", type=int, default=0,
462                         help="net-crossings rally-state gate on --preset (0=off; W2
463 Gate-A)")
464         ap.add_argument("--vs-min-crossings", type=int, default=0,
465                         help="net-crossings gate on --vs (default: same as --min-
466 crossings)")
467         ap.add_argument("--max-window-duration", type=float, default=None,
468                         help="override bounded-windowing cap (s) on BOTH presets (W1; e.g.
469 12)")
470         ap.add_argument("--vs-max-window-duration", type=float, default=None,
471                         help="cap (s) on --vs ONLY (default: same as --max-window-duration);
472 lets a "
473 "BASELINE (no cap) vs CANDIDATE (capped) promotion check run in
474 one shot")
475         ap.add_argument("--hard-cap", action="store_true",
476                         help="enforce the cap as a HARD cap on --preset (chop continuously-
477 active "
478 "windows at the cap when no inactivity gap exists; W1 hard-cap
479 fallback)")
480         ap.add_argument("--vs-hard-cap", action="store_true",
481                         help="hard-cap fallback on --vs (default: same as --hard-cap)")
482         ap.add_argument("--blob-fallback", action="store_true",
483                         help="R5 blob-fallback on --preset (after R4, force-chop any group
484 still over the "
485 "cap at its midpoint + flag/log it; the continuously-active
486 blob last resort)")
487         ap.add_argument("--vs-blob-fallback", action="store_true",
488                         help="R5 blob-fallback on --vs (default: same as --blob-fallback)")

```

```

478         ap.add_argument("--blob-factor", type=float, default=None,
479                         help="R5 magnitude gate (both presets): chop only groups > this ×
cap (default 4.0)")
480         ap.add_argument("--inactivity-end", type=float, default=None,
481                         help="R4 rally-end inactivity trim (s) on --preset (0=off; trims
post-rally drift tail)")
482         ap.add_argument("--vs-inactivity-end", type=float, default=None,
483                         help="R4 inactivity trim on --vs (default: same as --inactivity-
end)")
484         ap.add_argument("--inactivity-rally-thresh", type=float, default=None,
485                         help="R4 velocity above which motion counts as 'rally' (both
presets; default 0.08)")
486         ap.add_argument("--inactivity-min-density", type=float, default=None,
487                         help="R4 min trailing fast-fraction to stay 'in rally' (both
presets; default 0.34)")
488         ap.add_argument("--tau-start", type=float, default=None,
489                         help="R2 tolerance-match start window (s); default 2.0 (forgives the
service window)")
490         ap.add_argument("--tau-end", type=float, default=None,
491                         help="R2 tolerance-match end window (s); default 1.5 (keeps the
rally-end honest)")
492         ap.add_argument("--guard-weight-merge", type=float, default=None,
493                         help="R1 over-seg guard: merge cost weight (default 2.0; merge hides
a rally)")
494         ap.add_argument("--guard-weight-split", type=float, default=None,
495                         help="R1 over-seg guard: split cost weight (default 1.0; split only
fragments)")
496         ap.add_argument("--guard-counts", action="store_true",
497                         help="R1 over-seg guard scores raw counts instead of normalized
rates "
498                             "(rates are the default ? comparable across a structural
mega?bounded change)")
499         ap.add_argument("--min-duration", type=float, default=0.0,
500                         help="LONG-RALLY LENS (0=OFF): score ONLY rallies longer than this
many seconds, "
501                             "filtering BOTH predictions and golden GTs before matching.
Strips local's "
502                             "short false-positive blips to reveal long-rally quality.
Applies across the "
503                             "tIoU + R2 paths and to --vs (same filter on both sides).")
504         ap.add_argument("--stratify", action="store_true",
505                         help="also print the long-rally stratified report: {all vs
long(>--long-tau)} × "
506                             "{single vs multicourt} aggregates (court type from
GOLDEN_VIDEOS.md / the "
507                             "manifest court_type field). The actual long-rally payoff
table.")
508         ap.add_argument("--long-tau", type=float, default=LONG_RALLY_TAU,
509                         help=f"the 'long' rally duration threshold (s) used by --stratify "
510                             f"(default {LONG_RALLY_TAU:g}; owner's North-star happy-path is
>5s).")
511         ap.add_argument("--json-out", default=None, help="optional path to write the full
result JSON")
512         args = ap.parse_args()
513
514         golden = load_golden(args.manifest, args.features_dir)
515         base = _build_preset(args, "base")
516         if args.vs:
517             vs_mc = args.vs_min_crossings or args.min_crossings
518             cand = _build_preset(args, "cand")
519             result = compare(golden, base, cand, args.iou,
520                             base_min_crossings=args.min_crossings,
cand_min_crossings=vs_mc,
521                             tau_start=args.tau_start, tau_end=args.tau_end,
522                             guard_weight_merge=args.guard_weight_merge,

```

```

523             guard_weight_split=args.guard_weight_split,
524             guard_use_rates=not args.guard_counts,
525             min_duration=args.min_duration)
526     print(format_report(result["base"]))
527     print("\n" + format_report(result["cand"]))
528     d = result["delta"]
529     print(f"\n## ? ({args.vs} mc={vs_mc} ? {args.preset} mc={args.min_crossings}), "
530           f"\npaired={result['n_paired']}")
531     for k, dd in d.items():
532         st = dd["sign_test"]
533         print(f" ?{k}: {dd['mean_delta']:+.3f}
534               f"CI?0={dd['ci_excludes_zero']} sign(W/L)={st['wins']}/{st['losses']}
535               p={st['p_value']:.3f}")
536         print(_format_guard(result["over_seg_guard"]))
537     else:
538         result = evaluate(golden, base, args.iou, args.min_crossings,
539                           tau_start=args.tau_start, tau_end=args.tau_end,
540                           min_duration=args.min_duration)
541     print(format_report(result))
542     if args.stratify:
543         # The long-rally payoff table always reports the BASE preset (the served/current
544         # config),
545         # split by duration × court type. Independent of --min-duration (it does its own
546         # split).
547         print(format_stratified(stratified_report(
548             golden, base, args.iou, args.min_crossings,
549             tau_start=args.tau_start, tau_end=args.tau_end, long_tau=args.long_tau)))
550     if args.json_out:
551         with open(args.json_out, "w", encoding="utf-8") as fh:
552             json.dump(result, fh, indent=2)
553             print(f"\n[written {args.json_out}]")
554
555     if __name__ == "__main__":
556         main()

```

5. assistant (2026-06-22T09:48:53.838Z)

Now let me examine the specific lines mentioned in the finding to verify the claim:

6. user (2026-06-22T09:48:55.531Z)

```

365         if err2 is None and count2 is not None:
366             count, intervals = count2, intervals2 # the reproduced (or corrected)
367             value
368             quality = None
369             if gts and intervals is not None:
370                 row = rally_seg_eval.score_intervals(intervals, gts)
371                 quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
372                 "tol_precision", "tol_recall")}
373                 if k in row}
374             return {"name": name, "reel_count": count, "ok": True, "error": None,

```

7. user (2026-06-22T09:48:56.652Z)

```

75     # ----- #
76     # Pure scoring core (no I/O ? unit-testable)
77     # ----- #
78     def score_intervals(preds: List[Interval], gts: List[Interval],
79                         iou: float = DEFAULT_IOU,

```

```

80         tau_start: Optional[float] = None,
81         tau_end: Optional[float] = None,
82         min_duration: float = 0.0) -> Dict[str, Any]:
83     """The full per-video metric row for predicted vs ground-truth rally intervals.
84
85     Pairs temporal-IoU P/R/F1 + boundary error (`metrics.score`) with the
86     over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown)
87     so a
88     merged mega-window is *visible* even when IoU-F1 isn't moved.
89
90     Also carries the **R2** task-faithful block (`tolerance_metrics`, the headline
91     going forward;
92     tIoU here is the SECONDARY trend-line per the augment-then-ablation-gate decision):
93     strict-1:1
94     tolerance-match `tol_f1/tol_precision/tol_recall` (over-production costs
95     precision) + the
96     start/end boundary-error medians + the over-seg guard counts. See
97     docs/R2_EVAL_METRIC_DESIGN.md.
98
99     `min_duration` > 0 applies the **long-rally lens**
100     (:func:`filter_by_min_duration`): BOTH preds
101     and GTs are filtered to rallies longer than that many seconds BEFORE any matching,
102     so the whole
103     row (tIoU set AND R2 block) scores only long rallies. Default 0.0 = OFF (exact
104     passthrough).
105
106     """
107     from backend.eval import tolerance_metrics as tm
108     preds = filter_by_min_duration(preds, min_duration)
109     gts = filter_by_min_duration(gts, min_duration)
110     ts = tm.TAU_START if tau_start is None else tau_start
111     te = tm.TAU_END if tau_end is None else tau_end
112     sc = metrics.score(preds, gts, iou_threshold=iou)
113     f1k = segmentation_metrics.f1_at_overlaps(preds, gts)
114     ms = segmentation_metrics.merge_split_report(preds, gts)
115     tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
116     be = tm.boundary_error_report(preds, gts) # unconditional best-overlap (the
117     R4-aiming diagnostic)
118     osr = tm.over_seg_report(preds, gts)
119     return {
120         "f1": sc.f1, "precision": sc.precision, "recall": sc.recall, "mean_iou":
121         sc.mean_iou,
122         "mean_start_error": sc.mean_start_error, "mean_end_error": sc.mean_end_error,
123         "f1@0.1": f1k[0.1], "f1@0.25": f1k[0.25], "f1@0.5": f1k[0.5],
124         "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
125         "merge_rate": ms["merge_rate"], "split_rate": ms["split_rate"],
126         "merges": ms["merges"], "splits": ms["splits"], "missed": ms["missed"],
127         "spurious": ms["spurious"], "num_pred": len(preds), "num_gt": len(gts),
128         # --- R2 (tolerance-match) ? the task-faithful headline block ---
129         "tau_start": ts, "tau_end": te,
130         "tol_f1": tol_f1, "tol_precision": tol_p, "tol_recall": tol_r,
131         "tol_matched": float(len(tol_matches)),
132         "dstart_abs_median": be["dstart"]["abs_median"], "dend_abs_median":
133         be["dend"]["abs_median"],
134         "split_count": osr["split_count"], "merge_count": osr["merge_count"],
135     }

```

8. assistant (2026-06-22T09:48:58.706Z)

Perfect. Now let me verify the extraction logic is indeed safe and the keys match:

9. user (2026-06-22T09:49:00.750Z)

```

55:QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
201:         for m in QUALITY_HIGHER:

```

10. assistant (2026-06-22T09:49:03.054Z)

Excellent. Let me verify line 201 in context to see the quality extraction logic in the compare function:

11. user (2026-06-22T09:49:03.964Z)

```
195         # 2) reel count ? EXACT
196         bc, cc = b.get("reel_count"), c.get("reel_count")
197         if bc is not None and cc is not None and cc != bc:
198             res.findings.append(Finding(v, "reel_count", "regression", float(bc),
float(cc)))
199         # 3) quality metrics ? tolerance (only if both sides have them)
200         bq, cq = b.get("quality") or {}, c.get("quality") or {}
201         for m in QUALITY_HIGHER:
202             bv, cv = bq.get(m), cq.get(m)
203             if bv is None or cv is None:
204                 continue
205             if cv < bv - tols.quality_tol:
206                 res.findings.append(Finding(v, m, "regression", bv, cv))
207             elif cv > bv + tols.quality_tol:
208                 res.findings.append(Finding(v, m, "improvement", bv, cv))
209         return res
```

12. assistant (2026-06-22T09:49:06.541Z)

Now let me verify the actual extraction at line 371-372 one more time to confirm the keys being accessed:

13. user (2026-06-22T09:49:07.277Z)

```
368         quality = None
369         if gts and intervals is not None:
370             row = rally_seg_eval.score_intervals(intervals, gts)
371             quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
"tol_precision", "tol_recall")}
372             if k in row}
373
374         return {"name": name, "reel_count": count, "ok": True, "error": None,
375                "quality": quality, "wall_seconds": round(total_wall, 2)}
```

14. assistant (2026-06-22T09:49:21.353Z)

Perfect. Now let me trace through the logic to verify the claim:

Analysis

****Verification of the finding:****

- **`rally\_seg\_eval.score\_intervals()` return dict**** (lines 109-123 in rally_seg_eval.py):
 - Returns a dict with keys: `f1`, `precision`, `recall`, `mean\_iou`, `mean\_start\_error`, `mean\_end\_error`, `f1@0.1`, `f1@0.25`, `f1@0.5`, `segment\_count\_ratio`, `merge\_rate`, `split\_rate`, `merges`, `splits`, `missed`, `spurious`, `num\_pred`, `num\_gt`, `tau\_start`, `tau\_end`, `tol\_f1`, `tol\_precision`, `tol\_recall`, `tol\_matched`, `dstart\_abs\_median`, `dend\_abs\_median`, `split\_count`, `merge\_count`
- **Keys extracted at lines 371-372 in nightly_reelcount.py:****
 - The code attempts to extract: `tol\_f1`, `f1@0.5`, `f1@0.25`, `tol\_precision`, `tol\_recall`
 - The extraction uses a dict comprehension with ****`if k in row`**** guard clause
- **Verification of safety:****
 - ? `tol\_f1` ? ****present**** at line 119 of rally_seg_eval.py
 - ? `f1@0.5` ? ****present**** at line 112 of rally_seg_eval.py

- ? `f1@0.25` ? **present** at line 112 of rally_seg_eval.py
- ? `tol\_precision` ? **present** at line 119 of rally_seg_eval.py
- ? `tol\_recall` ? **present** at line 119 of rally_seg_eval.py

4. **Extraction logic (line 371-372):**

```

python
quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25", "tol_precision", "tol_recall")
           if k in row}
...

```

- This uses `if k in row` to conditionally include only keys that exist
- Missing keys are **silently skipped** (not added to the dict)
- A missing key would result in an empty `quality` dict or a partial one, **never a KeyError**

5. **Quality comparison logic (lines 200-208 in compare function):**

- Uses `.get(m)` on both baseline and current quality dicts
- Defaults to `None` if key missing
- Line 203: `if bv is None or cv is None: continue` ? explicitly handles missing keys gracefully

Correctness assessment:

- The finding claims extraction is "safe" ? this is **ACCURATE**
- All required keys are present in the `score\_intervals()` return dict
- The conditional dict comprehension (`if k in row`) prevents KeyError
- Missing keys degrade gracefully (empty/partial dicts, no crash)
- Downstream code handles `None` values correctly with `.get()` + explicit `None` checks

No defects found. This is production-safe code with proper defensive programming.

15. user (2026-06-22T09:49:26.841Z)

Structured output provided successfully

16. assistant (2026-06-22T09:49:31.079Z)

Verification complete. The finding is **REAL** and **ACCURATE**.

The code at `nightly\_reelcount.py:371-372` safely extracts quality metrics from the `score\_intervals()` return dict. All five keys (`tol\_f1`, `f1@0.5`, `f1@0.25`, `tol\_precision`, `tol\_recall`) are guaranteed to be present in the dict returned by `rally\_seg\_eval.py:109-123`, and the extraction uses a defensive dict comprehension with `if k in row` guard that silently skips any missing keys. Downstream quality comparison in the `compare()` function (lines 200-208) further protects against `None` values with explicit checks. **No defects, no regression risk.** This is production-safe code.